L03 — Asymptotic Notations: Big-O, Big-Ω, Big-Θ with Examples

Unit: 1 | **Chapter:** Fundamentals of Data Structures & Arrays | **BT Level:** BT4 | **CO:** CO1

Learning Objectives

- Define Big-O, Big-Ω, and Big-Θ notations mathematically
- Identify the asymptotic complexity of algorithms using each notation
- Simplify expressions using asymptotic rules
- Determine tight bounds for common algorithms

---

1. Why Asymptotic Analysis?

When we say an algorithm takes $3n^2 + 5n + 7$ operations, the dominant term for large n is n^2. Constants and lower-order terms become insignificant.

Asymptotic analysis describes the growth rate of a function as $n \to \infty$, ignoring:

- Constant coefficients (hardware-dependent)
- Lower-order terms (irrelevant for large n)

---

2. Big-O Notation — Upper Bound

Definition: $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \leq c \cdot g(n) \quad \forall , n \geq n_0$$

Interpretation: g(n) is an **upper bound** on the growth of f(n). The algorithm runs **at most** this fast.

2.1 Examples

f(n)	Big-O	Reasoning
$3n + 5$	$O(n)$	$3n + 5 \leq 4n$ for $n \geq 5$
$2n^2 + 3n$	$O(n^2)$	$2n^2 + 3n \leq 3n^2$ for $n \geq 3$
5	$O(1)$	constant, no growth
$n \log n + n$	$O(n \log n)$	$n \log n$ dominates
$n^3 + 2^n$	$O(2^n)$	2^n dominates for large n

2.2 Formal Verification Example

Prove that $f(n) = 4n + 2$ is $O(n)$:

Choose $c = 5$, $n_0 = 2$:

- For $n \geq 2$: $4n + 2 \leq 4n + n = 5n$ ✓

So $c = 5$, $n_0 = 2$ satisfies the definition.

---

3. Big-Ω Notation — Lower Bound

Definition: $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \geq c \cdot g(n) \quad \forall , n \geq n_0$$

Interpretation: g(n) is a **lower bound** on the growth of f(n). The algorithm takes **at least** this long.

3.1 Examples

f(n)	Big-Ω	Reasoning
$3n^2 + 1$	$\Omega(n^2)$	$3n^2 + 1 \geq 3n^2$ for all $n \geq 0$
$n \log n$	$\Omega(n)$	$n \log n \geq n$ for $n \geq 2$
$n!$	$\Omega(2^n)$	$n!$ grows faster than 2^n

3.2 Use Case

Ω notation is used to show **algorithmic lower bounds** — for example, any comparison-based sorting algorithm requires at least Ω(n log n) comparisons in the worst case.

---

4. Big-Θ Notation — Tight Bound

Definition: $f(n) = \Theta(g(n))$ if:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall , n \geq n_0$$

Interpretation: g(n) is a **tight bound**. The algorithm grows **exactly as fast** as g(n) (up to constant factors).

$f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ **and** $f(n) = \Omega(g(n))$

4.1 Examples

f(n)	Big-Θ	Reasoning
$2n^2 + 5n$	$\Theta(n^2)$	Bounded above and below by n^2
$n/2$	$\Theta(n)$	Bounded above and below by n
$\log_2(n) + 3$	$\Theta(\log n)$	Bounded above and below by log n

4.2 Formal Verification Example

Prove `f(n) = 2n + 3` is `Θ(n)`:

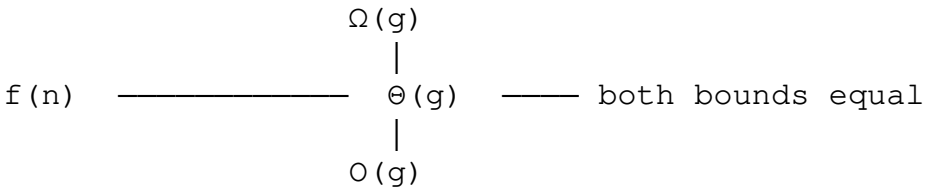
- **Upper bound:** $2n + 3 \leq 3n$ for $n \geq 3$, so $f(n) = O(n)$ with $c_2 = 3$
- **Lower bound:** $2n + 3 \geq 2n$ for all $n \geq 0$, so $f(n) = \Omega(n)$ with $c_1 = 2$

Since both hold: $f(n) = \Theta(n)$ ✓

---

5. Summary of Notations

Notation	Meaning	Use Case
O(g)	Upper bound — "at most"	Worst-case analysis
Ω(g)	Lower bound — "at least"	Best-case analysis, proving lower bounds
Θ(g)	Tight bound — "exactly"	Average/typical behavior



---

6. Asymptotic Simplification Rules

Rule 1: Drop Constants

$O(5n) \to O(n), O(100) \to O(1)$

Rule 2: Drop Lower-Order Terms

$O(n^2 + n) \to O(n^2), O(n \log n + n) \to O(n \log n)$

Rule 3: Dominant Term

$O(n^3 + n^2 + n) \to O(n^3)$

Rule 4: Logarithm Base

$O(\log_2 n) = O(\log_{10} n) = O(\ln n)$ — all equivalent (differ by constant factor)

Rule 5: Sum Rule

If $T_1 = O(f)$ and $T_2 = O(g)$, then $T_1 + T_2 = O(\max(f, g))$

Rule 6: Product Rule

If $T_1 = O(f)$ and $T_2 = O(g)$, then $T_1 \times T_2 = O(f \times g)$

---

7. Worked Examples

Example 1: Nested Loop

```
for i in range(n):          # O(n)
    for j in range(n):      # O(n)
        print(i, j)         # O(1)
# Total: O(n × n × 1) = O(n²)
```

Example 2: Sequential Loops

```
for i in range(n):          # O(n)
    print(i)

for j in range(n):          # O(n)
    for k in range(n):      # O(n)
        print(j, k)
# Total: O(n) + O(n²) = O(n²)  [Sum Rule → take max]
```

Example 3: Halving Loop

```
i = n
while i > 0:    # How many times can we halve n before reaching 0?
    i = i // 2  # i = n, n/2, n/4, ..., 1  → log₂(n) iterations
# Total: O(log n)
```

Example 4: Identify complexity of $f(n) = 6n^3 + 4n^2 + 2n + 100$

- Dominant term: n^3
- Big-O: $O(n^3)$, Big-Ω: $\Omega(n^3)$, Big-Θ: $\Theta(n^3)$

---

8. Little-o and Little-ω (Brief Overview)

- **o(g):** Strictly less than g — f grows **strictly slower** than g (e.g., $n = o(n^2)$)
- **ω(g):** Strictly greater than g — f grows **strictly faster** than g (e.g., $n^2 = \omega(n)$)

These are used less frequently but appear in theoretical analysis.

Summary

Notation	Bound Type	Formal Condition
$O(g)$	Upper	$f(n) \leq c \cdot g(n)$ for $n \geq n_0$
$\Omega(g)$	Lower	$f(n) \geq c \cdot g(n)$ for $n \geq n_0$
$\Theta(g)$	Tight	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

- Drop constants and lower-order terms.
- Use the dominant term to determine complexity class.
- Big-Θ is the most precise — use it when both bounds match.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 3 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 2.2 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 2.1 (Springer, 2024)